Lab02 的实验报告

刘田田 25300120181

1. phase1

(1) 关键汇编片段

Phase_01 部分主要是先检验输入是否合法，不合法直接对应失败

0x000000000040209d <+13>: mov $0x1,%esi

0x00000000004020aa <+26>: call 0x402ca0 <valid_u32_fields>

0x00000000004020af <+31>: test %al,%al

0x00000000004020b1 <+33>: jne 0x4020c0 <phase_01+48>

0x00000000004020b7 <+39>: xor %eax,%eax

0x00000000004020bd <+45>: ret

然后调用标准库函数 sscanf，按%u 格式将字符串转变成无符号整数存入栈中，解析失败则跳转到失败路径。

0x00000000004020c2 <+50>: lea -0x14(%rbp),%rdx

0x00000000004020c6 <+54>: mov $0x403189,%esi

0x00000000004020ce <+62>: call 0x4011e0 <__isoc99_sscanf@plt>

0x00000000004020d3 <+67>: cmp $0x1,%eax

0x00000000004020d6 <+70>: jne 0x4020b3 <phase_01+35>

传入 challenge 种子进行 p1_compute 三级运算，将输出结果与用户输入的标准答案进行比较，相等则返回真，通关。

0x00000000004020d8 <+72>: mov %r12,%rdi

0x00000000004020db <+75>: call 0x402050 <p1_compute>

0x00000000004020e0 <+80>: cmp %eax,-0x14(%rbp)

0x00000000004020e3 <+83>: sete %al

0x00000000004020ee <+94>: ret

其中 p1_compute 通过 p1_step0,1,2 进行运算。P1_compute 先将种子拆成高 32 位、右移 16 位中间值、低 32 位，作为后续子函数的输入参数。

0x000000000040205b <+11>: mov %edi,%r8d

0x000000000040205e <+14>: shr $0x20,%rax

0x0000000000402062 <+18>: shr $0x10,%rdx

依次调用三级子函数，前一步结果作为后一步的输入，同时备份 step0 结果用于合并

0x000000000040206e <+30>: call 0x402020 <p1_step0>

0x0000000000402079 <+41>: mov %eax,%ecx

0x000000000040207b <+43>: call 0x402030 <p1_step1>

0x0000000000402085 <+53>: call 0x402040 <p1_step2>

Step0 的步骤是高 32 位加常量，再与低 32 位异或

0x0000000000402024 <+4>: lea 0xfe31(%rsi),%eax
0x000000000040202a <+10>: xor %edi,%eax
0x000000000040202c <+12>: ret
Step1 的步骤是高 32 位乘 3，再与偏移值加常量的结果异或
0x0000000000402034 <+4>: lea (%rdi,%rdi,2),%eax
0x0000000000402037 <+7>: add $0x2a60,%esi
0x000000000040203d <+13>: xor %esi,%eax
0x000000000040203f <+15>: ret
Step2 的步骤是 step1 结果乘 3 加低 32 位，再与常量异或
0x0000000000402044 <+4>: lea (%rdi,%rdi,2),%eax
0x0000000000402047 <+7>: add %esi,%eax
0x0000000000402049 <+9>: xor $0x28c0,%eax
0x000000000040204e <+14>: ret
将 step2 运算结果与备份的 step0 结果按位异或，得到本关最终的正确整数答案。
0x000000000040208b <+59>: xor %ecx,%eax
0x000000000040208d <+61>: ret

(2) 计算关系恢复与推理过程

首先，调用 valid_u32_fields 时第二个参数传入常量 1，说明函数要求输入恰好包含 1 个合法的 32 位无符号整数；其次，sscanf 执行后将返回值与 1 比较，说明程序只期望解析出 1 个整数字段。两处逻辑互相印证，确定本关答案为单个 32 位无符号整数。通过逐行分析 p1_compute 与三级子函数的汇编指令，还原出完整的计算流程：

先将 64 位 challenge 拆分为三个 32 位片段：高 32 位 high32、低 32 位 low32、右移 16 位中间值 shift16；step0 运算。step0 = (high32 + 0xfe31) ^ low32，step1 运算：step1 = (3 * high32) ^ (shift16 + 0x2a60)，step2 运算：step2 = (3 * step1 + low32) ^ 0x28c0，最终合并得到答案：answer = step2 ^ step0

(3) 解法代码的实现

```c
int solve_phase_1(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;

    uint32_t high32 = (uint32_t)(challenge >> 32);
    uint32_t low32 = (uint32_t)challenge;
    uint32_t shift16 = (uint32_t)(challenge >> 16);

    uint32_t step0_res = (high32 + 0xfe31) ^ low32;
    uint32_t step1_res = (3 * high32) ^ (shift16 + 0x2a60);
    uint32_t step2_res = (3 * step1_res + low32) ^ 0x28c0;

    values[0] = step2_res ^ step0_res;
    count = 1;
    return write_answer(out, cap, values, count);
}
```

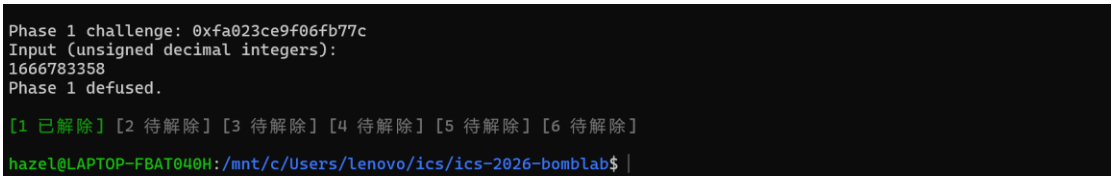
(4) 自定义 seed 验证记录

先运行./bomb --seed 123 --phase 1 获得当 seed=123 时的结果 1666783358

跟根据公式算出来的数字进行比较，相同，然后运行程序，炸弹被拆除，通过。答案正确

(5) 运行结果





2.phase2

(1)关键汇编片段

先通过调用的 valid_u32_fields 时第二个参数传 6 说明要求输入 6 位合法 32 位整数，sscanf 执行之后必须成功解析 6 个片段才允许通过。

0x00000000004020fd <+13>: mov $0x6,%esi

0x000000000040210c <+28>: call 0x402ca0 <valid_u32_fields>

0x0000000000402111 <+33>: test %al,%al

0x0000000000402113 <+35>: jne 0x402128 <phase_02+56>

0x000000000040214e <+94>: call 0x4011e0 <__isoc99_sscanf@plt>

0x0000000000402155 <+101>: cmp $0x6,%eax

0x0000000000402158 <+104>: jne 0x402115 <phase_02+37>

然后直接将 challenge 右移 13 位再与 0x7F 做与计算，提取最低七位，结果必须与输入的第一个数相等

0x000000000040215a <+106>: mov %r13,%rax

0x000000000040215d <+109>: mov -0x40(%rbp),%esi

```
0x0000000000402160 <+112>:    shr    $0xd,%rax
0x0000000000402164 <+116>:    and    $0x7f,%eax
0x0000000000402167 <+119>:    cmp    %eax,%esi
0x0000000000402169 <+121>:    jne    0x402115 <phase_02+37>
```

接着以此循环校验第 2 到 6 个数，每次循环 challenge 右移位数加三，challenge 先右移本轮循环需要的位数，再通过与计算取出低五位数值，加上当前序号后乘三，再与当前输入的第 i 个数相比较，不等就失败，然后进入下一轮。

```
0x000000000040216b <+123>:    mov    $0x7,%ecx
0x0000000000402170 <+128>:    mov    $0x1,%edx
0x0000000000402175 <+133>:    mov    %r13,%rax
0x0000000000402178 <+136>:    shr    %cl,%rax
0x000000000040217b <+139>:    and    $0x1f,%eax
0x000000000040217e <+142>:    add    %edx,%eax
0x0000000000402180 <+144>:    xor    %esi,%eax
0x0000000000402182 <+146>:    mov    -0x40(%rbp,%rdx,4),%esi
0x0000000000402186 <+150>:    lea    (%rax,%rax,2),%eax
0x0000000000402189 <+153>:    cmp    %eax,%esi
0x000000000040218b <+155>:    jne    0x402115 <phase_02+37>
0x000000000040218d <+157>:    add    $0x1,%rdx
0x0000000000402191 <+161>:    add    $0x3,%ecx
0x0000000000402194 <+164>:    cmp    $0x6,%rdx
0x0000000000402198 <+168>:    jne    0x402175 <phase_02+133>
```

(2)计算关系恢复与推理过程

首先第 1 个整数直接从 challenge 里提取固定位段得到，然后通过 challenge 位段值加上当前序号和前一个数做移位、加法、异或、乘 3 运算递推得到第 2-6 个整数，最后程序用循环逐个数比对，全部相等才通关。

(3)解法代码的实现

```c
int solve_phase_2(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;

    values[0] = (uint32_t)((challenge >> 13) & 0x7F);

    for (int i = 1; i < 6; ++i) {
        uint32_t shift = 7 + 3 * (i - 1);
        uint32_t tmp = (uint32_t)((challenge >> shift) & 0x1F);
        tmp += i;
        tmp ^= values[i-1];
        values[i] = tmp * 3;
    }

    count = 6;
    return write_answer(out, cap, values, count);
}
```

(4)自定义 seed 验证记录

这是当 seed = 42 时的验证记录，首先先用 defuser 指令得到 6 个数字，先于带入公式计算出来的序列进行比对，相同之后，然后进入炸弹程序验证

(5)运行结果

```
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./defuser --seed 42 --phase 2
124 297 864 2685 8010 24036
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ gdb ./bomb
```

```
hazel@LAPTOP-FBAT040H: /m
                    \
                  .-"""-.
                .'       '.
               /    LOOP    \
              |               |
              \               /
               '-----------'

[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
124 297 864 2685 8010 24036
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

[Inferior 1 (process 17375) exited normally]
(gdb)
```

3.phase3

(1)关键汇编片段

先通过调用 valid_u32_fields 和 sscanf 确定输入为两个整数，不合法返回失败

0x00000000004022cd <+13>: mov $0x2,%esi

0x00000000004022da <+26>: call 0x402ca0 <valid_u32_fields>

0x00000000004022df <+31>: test %al,%al

```
0x00000000004022e1 <+33>:    jne    0x4022f0 <phase_03+48>
0x00000000004022e3 <+35>:    xor    %eax,%eax
0x00000000004022ed <+45>:    ret
0x00000000004022f2 <+50>:    lea    -0x14(%rbp),%rcx
0x00000000004022f6 <+54>:    lea    -0x18(%rbp),%rdx
0x00000000004022fa <+58>:    mov    $0x403186,%esi
0x00000000004022ff <+63>:    mov    %rbx,%rdi
0x0000000000402302 <+66>:    call   0x4011e0 <__isoc99_sscanf@plt>
0x0000000000402307 <+71>:    cmp    $0x2,%eax
0x000000000040230a <+74>:    jne    0x4022e3 <phase_03+35>
```

然后检验第一个数

```
0x000000000040230c <+76>:    mov    %r12d,%edi
0x0000000000402311 <+81>:    and    $0x7,%edi
0x0000000000402314 <+84>:    cmp    %edi,-0x18(%rbp)
0x0000000000402317 <+87>:    jne    0x4022e5 <phase_03+37>
```

计算所有分支共用的运算参数，调用分支分发函数得到结果，与输入的第二个数比较，相等则通关。

```
0x0000000000402319 <+89>:    mov    %r12,%rax
0x000000000040231c <+92>:    mov    %r12,%rsi
0x000000000040231f <+95>:    shr    $0x3,%rax
0x0000000000402323 <+99>:    shr    $0x20,%rsi
0x0000000000402327 <+103>:   xor    %eax,%esi
0x0000000000402329 <+105>:   call   0x402260 <p3_dispatch>
0x000000000040232e <+110>:   cmp    %eax,-0x14(%rbp)
0x0000000000402331 <+113>:   sete   %al
0x0000000000402334 <+116>:   jmp    0x4022e5 <phase_03+37>
```

做范围检查，索引大于 7 则越界返回 0；否则通过数据段跳转表间接跳转到对应分支函数。

```
0x0000000000402264 <+4>:     mov    %edi,%eax
0x0000000000402266 <+6>:     mov    %esi,%edi
0x0000000000402268 <+8>:     cmp    $0x7,%eax
0x000000000040226b <+11>:    ja     0x4022b5 <p3_dispatch+85>
0x000000000040226d <+13>:    notrack jmp *0x405820(,%rax,8)
```

(2)计算关系恢复与推理过程

首先确定输入为两个合法 32 位无符号整数，然后通过 sscanf 解析为整数入栈，解析失败则直接返回。然后通过取 challenge 的低 32 位通过与计算保留最低 3 位，再将结果与输入的第一个数比较，相等则通过，不相等就失败的方式校验第一个数。计算所有分支公用的运算参数，再做范围检查，索引大于 7 则越界返回零，否

则通过数据段跳转到对应分支函数。接着，程序将 challenge 右移 32 位的高 32 位，与 challenge 右移 3 位的低 32 位做异或运算，得到所有分支共用的输入参数。调用分支分发函数，将返回值与第二个输入数比对，相等则通关。其中每个分支函数公式如下：

v0=0：v1 = 5 * arg + 0x821b

v0=1：v1 = 5 * arg + 0x29a1d

v0=2：v1 = (arg ^ 0xd8e5) + (arg >> 2)

v0=3：v1 = 5 * arg + 0xc5ad

v0=4：v1 = (4 * arg) ^ 0x2ea2

v0=5：v1 = (arg ^ 0x8fb8) + (arg >> 2)

v0=6：v1 = 5 * arg + 0x3c186

v0=7：v1 = (4 * arg) ^ 0xb962

(3)解法代码的实现

```
int solve_phase_3(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};

    uint32_t arg = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3);

    uint32_t v1 = 0;
    switch(v0) {
        case 0:
            v1 = 5 * arg + 0x821b;
            break;
        case 1:
            v1 = 5 * arg + 0x29a1d;
            break;
        case 2:
            v1 = (arg ^ 0xd8e5) + (arg >> 2);
            break;
        case 3:
            v1 = 5 * arg + 0xc5ad;
            break;
        case 4:
            v1 = (4 * arg) ^ 0x2ea2;
            break;
        case 5:
            v1 = (arg ^ 0x8fb8) + (arg >> 2);
            break;
        case 6:
            v1 = 5 * arg + 0x3c186;
            break;
        case 7:
            v1 = (4 * arg) ^ 0xb962;
            break;
    }
    values[1] = v1;

    count = 2;
    return write_answer(out, cap, values, count);
}
```

(4)自定义 seed 验证记录

先利用 ./defuser –seed 00 –phase 3 时得到 seed = 00 时的值，然后将该值与代入公式后计算得到的值比较，若相同则带入炸弹程序检测

(5)运行结果

```
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./defuser --s
eed 00 --phase 3
4 3674770678
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./bomb --seed
 00 --phase 3
```



4.phase4

(1)关键汇编片段

检验输入是否是 1 个合法 32 位无符号函数，格式不合法直接返回失败，再用 sscanf 将输入字符串解析为 1 个无符号整数存入栈中，解析失败则跳转失败路径

```
0x00000000004023bd <+13>:    mov    $0x1,%esi
0x00000000004023ca <+26>:    call   0x402ca0 <valid_u32_fields>
0x00000000004023cf <+31>:    test   %al,%al
0x00000000004023d1 <+33>:    jne    0x4023e0 <phase_04+48>
0x00000000004023d7 <+39>:    xor    %eax,%eax
0x00000000004023dd <+45>:    ret
0x00000000004023e2 <+50>:    lea    -0x14(%rbp),%rdx
0x00000000004023e6 <+54>:    mov    $0x403189,%esi
0x00000000004023eb <+59>:    mov    %rbx,%rdi
0x00000000004023ee <+62>:    call   0x4011e0 <__isoc99_sscanf@plt
0x00000000004023f3 <+67>:    cmp    $0x1,%eax
0x00000000004023f6 <+70>:    jne    0x4023d3 <phase_04+35>
```

从 challenge 提取四个参数传入 p4_walk 计算标准答案

```
0x00000000004023fe <+78>:    mov    %r12d,%edi
0x0000000000402401 <+81>:    xor    %esi,%esi
0x000000000040240b <+91>:    and    $0x7f,%edi
0x000000000040240e <+94>:    mov    $0x7f,%edx
0x0000000000402407 <+87>:    shr    $0x7,%rcx
0x0000000000402403 <+83>:    shr    $0x20,%rax
0x0000000000402413 <+99>:    xor    %eax,%ecx
0x0000000000402415 <+101>:   call   0x402340 <p4_walk>
0x000000000040241a <+106>:   cmp    %eax,-0x14(%rbp)
0x000000000040241d <+109>:   sete   %al
```

P4_walk 递归结构

```
0x0000000000402348 <+8>:     sub    %esi,%edi
0x000000000040234a <+10>:    shr    $1,%edi
0x000000000040234c <+12>:    add    %esi,%edi
0x000000000040234e <+14>:    cmp    %eax,%edi
0x0000000000402350 <+16>:    je     0x4023a0 <p4_walk+96>

0x00000000004023a0 <+96>:    lea    0x2b(%rdi),%eax
0x00000000004023a3 <+99>:    xor    %ecx,%eax
0x00000000004023a5 <+101>:   ret

0x0000000000402362 <+34>:    lea    0x1(%rdi),%esi
0x0000000000402367 <+39>:    call   p4_walk
0x000000000040236c <+44>:    lea    (%rax,%rax,2),%eax
0x000000000040236f <+47>:    xor    %ebx,%eax
0x0000000000402376 <+54>:    xor    $0xf,%eax

0x0000000000402380 <+64>:    lea    -0x1(%rdi),%edx
0x0000000000402385 <+69>:    call   p4_walk
0x000000000040238a <+74>:    lea    (%rax,%rax,2),%eax
0x000000000040238d <+77>:    xor    %ebx,%eax
0x0000000000402394 <+84>:    xor    $0x12,%eax
```

(2)计算关系恢复与推理过程

先确定输入为 1 位 32 位无符号整数，然后从 challenge 提取四个参数传入 p4_walk 计算标准答案，接着再 p4_walk 里面算出四个参数答案。Challenge 四个参数的计算方法是：通过与计算提取最低七位作为 challenge 目标值，low 为二分左边界，初始值为

0，high 为二分右边界初始值为 0x7F，x 为固定运算参数，x = (uint32_t)(challenge >> 7) ^ (uint32_t)(challenge >> 32)。在 p4_walk 里面，计算中点 mid = low + (high - low) / 2 。当 n == mid 时，直接返回 (mid + 0x2b) ^ x ，当 n > mid 时，递归调用右半区间，返回值(3 * 递归结果 ^ (mid + x)) ^ 0xf ，当 n < mid 时，递归调用左半区间，返回值做 (3 * 递归结果 ^ (mid + x)) ^ 0x12 运算

(3)解法代码的实现

```
static uint32_t p4_walk(uint32_t n, uint32_t low, uint32_t high, uint32_t x) {
    uint32_t mid = low + (high - low) / 2;
    if (n == mid) {
        return (mid + 0x2b) ^ x;
    } else if (n > mid) {
        uint32_t res = p4_walk(n, mid + 1, high, x);
        return (3 * res ^ (mid + x)) ^ 0xf;
    } else {
        uint32_t res = p4_walk(n, low, mid - 1, x);
        return (3 * res ^ (mid + x)) ^ 0x12;
    }
}

int solve_phase_4(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;

    uint32_t n = (uint32_t)(challenge & 0x7F);
    uint32_t x = (uint32_t)(challenge >> 7) ^ (uint32_t)(challenge >> 32);

    values[0] = p4_walk(n, 0, 0x7F, x);
    count = 1;

    return write_answer(out, cap, values, count);
}
```

(4)自定义 seed 验证记录

先利用 ./defuser –seed 42 –phase 4 时得到 seed = 42 时的值，然后将该值与代入公式后计算得到的值比较，若相同则带入炸弹程序检测

(5)运行结果

```
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./defuser --s
eed 42 --phase 4
1434958806
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./bomb --phas
e 42 --phase 4
Invalid, duplicate, or missing option. Use --help.
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./bomb --seed
 42 --phase 4
──────────────────────────────────────────────────────────────────────────────
BOMBLAB 2026                                          这里还没有明天

                              *
                               \
                          .-""""-.
                        .'        '.
                       /    LOOP    \
                      |              |
                       \            /
                        '----------'

[1 待解除] [2 待解除] [3 待解除] [4 当前] [5 待解除] [6 待解除]

──────────────────────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x09bc585a244823f2
Input (unsigned decimal integers):
1434958806
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]
```

5.phase5

(1)关键汇编片段

首先检验输入是否为 2 个合法 32 位无符号整数，不合法直接返回失败，然后用 sscanf 将输入字符串解析为 2 个无符号整数存入栈中，解析失败则返回失败

```
0x00000000004024ad <+13>:    mov     $0x2,%esi
0x00000000004024ba <+26>:    call    0x402ca0 <valid_u32_fields>
0x00000000004024bf <+31>:    test    %al,%al
0x00000000004024c1 <+33>:    jne     0x4024d0 <phase_05+48>
0x00000000004024c3 <+35>:    xor     %eax,%eax
0x00000000004024cd <+45>:    ret
0x00000000004024d2 <+50>:    lea     -0x18(%rbp),%rcx
0x00000000004024d6 <+54>:    lea     -0x1c(%rbp),%rdx
0x00000000004024da <+58>:    mov     $0x403186,%esi
```

```
0x00000000004024df <+63>:    mov    %rbx,%rdi
0x00000000004024e2 <+66>:    call   0x4011e0 <__isoc99_sscanf@plt>
0x00000000004024e7 <+71>:    cmp    $0x2,%eax
0x00000000004024ea <+74>:    jne    0x4024c3 <phase_05+35>
```

传入 challenge 种子和输入指针，调用 p5_traverse 完成链表遍历计算，返回值对应第一个数，指针传入值对应第二个数，全部匹配则通关。

```
0x00000000004024ec <+76>:    lea    -0x14(%rbp),%rsi
0x00000000004024f0 <+80>:    mov    %r12,%rdi
0x00000000004024f3 <+83>:    call   0x402430 <p5_traverse>
0x00000000004024f8 <+88>:    mov    %eax,%edx
0x00000000004024fc <+92>:    cmp    %edx,-0x1c(%rbp)
0x00000000004024ff <+95>:    jne    0x4024c5 <phase_05+37>
0x0000000000402501 <+97>:    mov    -0x14(%rbp),%eax
0x0000000000402504 <+100>:   cmp    %eax,-0x18(%rbp)
0x0000000000402507 <+103>:   sete   %al
```

这是 p5_traverse 的遍历汇编

```
0x000000000040243d <+13>:    and    $0x7,%eax
0x0000000000402444 <+20>:    mov    0x405b60(,%rax,8),%rdx
0x000000000040245e <+46>:    xor    $0xeaa5,%eax
0x0000000000402463 <+51>:    lea    (%rax,%rax,2),%eax
0x0000000000402466 <+54>:    add    0xc(%rdx),%eax
0x000000000040246c <+60>:    xor    %esi,%eax

0x0000000000402478 <+72>:    mov    (%rdx),%rdx
0x000000000040247b <+75>:    lea    (%rax,%rax,2),%eax
0x0000000000402484 <+84>:    add    0xc(%rdx),%eax
0x0000000000402487 <+87>:    xor    %esi,%eax
0x000000000040248c <+92>:    cmp    %ecx,%edi
0x000000000040248e <+94>:    jae    0x402478 <p5_traverse+72>
```

(2)计算关系恢复与推理过程

先通过 valid_u32_fields 和 sscanf 判断出答案为 2 个 32 位无符号整数，然后结合调用前的寄存器赋值，还原出 p5_traverse 的两个输入分别为 challenge 种子和 32 位整数指针，返回一个对应第一个整数的返回值和对应第二个整数的指针返回值。函数通过 challenge 最低 3 位作为索引，从全局数组 p5_heads 中选中对应链表的头节点；每个节点包含 next 指针、val_a、val_b 三个字段，通过 next 指针串联成链。将 challenge 右移的低 32 位与 challenge 高 32 位异或，再异或常量 0xeaa5 得到初始运算基数；初始基数乘 3 加头节点 val_b，再异或头节点 val_a，得到第一轮运算结果。

其中，遍历次数由 challenge 右移 3 位的低 2 位决定，范围为 0~3 次。每向后遍历一个节点，执行 结果 = (结果 * 3 + 当前节点.val_b) ^ 当前节点.val_a 的运算，同时更新第二个输出值为当前节点的 val_a。遍历结束后，迭代结果为第一个正确答案，最后一个节点的 val_a 为第二个正确答案。

(3)解法代码的实现

```
typedef struct p5_node {
    const struct p5_node *next;
    uint32_t val_a;
    uint32_t val_b;
} p5_node_t;

static const p5_node_t nodes[] = {
    /* 0x405c40 */ { (const p5_node_t*)0x405c20, 0x096a, 0xec28 },
    /* 0x405c20 */ { (const p5_node_t*)0x405bf0, 0x050b, 0x002a },
    /* 0x405bf0 */ { (const p5_node_t*)0x405ba0, 0x0222, 0x78b1 },
    /* 0x405ba0 */ { (const p5_node_t*)0x405be0, 0x081a, 0xdc28 },
    /* 0x405be0 */ { (const p5_node_t*)0x405bc0, 0x0671, 0xb1f3 },
    /* 0x405bc0 */ { (const p5_node_t*)0x405c10, 0x040b, 0xcd42 },
    /* 0x405c10 */ { (const p5_node_t*)0x405c50, 0x0028, 0x4f4c },
    /* 0x405c50 */ { (const p5_node_t*)0x405c00, 0x0336, 0xae87 },
    /* 0x405c00 */ { (const p5_node_t*)0x405c30, 0x0151, 0x7c49 },
    /* 0x405c30 */ { (const p5_node_t*)0x405bd0, 0x0a3f, 0xc7aa },
    /* 0x405bd0 */ { (const p5_node_t*)0x405bb0, 0x0b17, 0x0d26 },
};

static const p5_node_t *const p5_heads[8] = {
    &nodes[0],  // idx=0, 0x405c40
    &nodes[1],  // idx=1, 0x405c20
    &nodes[2],  // idx=2, 0x405bf0
    &nodes[3],  // idx=3, 0x405ba0
    &nodes[4],  // idx=4, 0x405be0
    &nodes[5],  // idx=5, 0x405bc0
    &nodes[6],  // idx=6, 0x405c10
    &nodes[7],  // idx=7, 0x405c50
};

static uint32_t p5_traverse(uint64_t challenge, uint32_t *out_v1) {
    uint32_t idx = (uint32_t)(challenge & 0x7);
    const p5_node_t *cur = p5_heads[idx];

    uint32_t base = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
    base ^= 0xeaa5;
    base = base * 3 + cur->val_b;
    base ^= cur->val_a;
    *out_v1 = cur->val_a;
```

```
    uint32_t loop_cnt = (uint32_t)((challenge >> 3) & 0x3);
    if (loop_cnt == 0) {
        return base;
    }

    for (uint32_t i = 1; i <= loop_cnt; ++i) {
        cur = cur->next;
        base = base * 3 + cur->val_b;
        base ^= cur->val_a;
        *out_v1 = cur->val_a;
    }

    return base;
}

int solve_phase_5(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;

    uint32_t v1;
    values[0] = p5_traverse(challenge, &v1);
    values[1] = v1;
    count = 2;

    return write_answer(out, cap, values, count);
}
```

(4)自定义 seed 验证记录

先利用 ./defuser –seed 42 –phase 5 时得到 seed = 42 时的值，然后将该值与代入公式后计算得到的值比较，若相同则带入炸弹程序检测

(5)运行结果

```
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./defuser --s
eed 42 --phase 5
2753877965 40
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./bomb --seed
 42 --phase 5
```



6.phase6

(1)关键汇编片段

传入 challenge 调用两级分发函数 p6_dispatch，若返回值与输入整数相等则通过。

```
0x0000000000402638 <+72>:    mov     %r12,%rdi
0x000000000040263b <+75>:    call    0x4025a0 <p6_dispatch>
0x0000000000402640 <+80>:    cmp     %eax,-0x14(%rbp)
0x0000000000402643 <+83>:    sete    %al
0x000000000040264e <+94>:    ret
```

以下是 p6_dispatch 的关键汇编

```
0x00000000004025ac <+12>:    shr     $0x6,%rdi
0x00000000004025b3 <+19>:    shr     $0x20,%rax
0x00000000004025b7 <+23>:    xor     %eax,%edi
0x00000000004025c3 <+35>:    and     $0x7,%eax
0x00000000004025c0 <+32>:    and     $0x38,%ebx

0x00000000004025c6 <+38>:    call    *0x405e00(,%rax,8)
```

0x00000000004025cd <+45>: xor $0x5746,%eax

0x00000000004025d4 <+52>: call *0x405e00(%rbx)

0x00000000004025df <+63>: add $0x9b24,%eax
0x00000000004025e4 <+68>: ret

(2)计算关系恢复与推理过程

先通过 valid_u32_fields 和 sscanf 判断答案为 1 个 32 位无符号整数。将 challenge 右移 6 位与 challenge 右移 32 位进行异或计算得到第一级运算的输入参数。先以 challenge 最低 3 位为索引，选中运算函数对初始参数做处理，然后将第一级结果与常量 0x5746 异或，作为第二级的输入，接着以 challenge 第 3~5 位为索引，选中运算函数对中间结果做处理，最后给第二级结果加常量 0x9b24，得到最终答案。

处理第二级输入函数分别为：

idx=0：res = (x 循环左移 3 位) ^ 0x6bc9

idx=1：res = ((x >> 7) ^ x) + 0x2aaf

idx=2：res = (x + 0xfcfd) ^ (x >> 7)

idx=3：res = (x ^ 0xd890) + (x << 7)

idx=4：res = ((x << 5) + 0xebab) ^ (3 * x)

idx=5：res = x 循环右移 3 位

idx=6：与 idx=3 复用同一分支

idx=7：与 idx=2 复用同一分支

(3)解法代码的实现

```c
int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;

    uint32_t idx1 = (uint32_t)(challenge & 0x7);
    uint32_t idx2 = (uint32_t)((challenge >> 3) & 0x7);

    uint32_t arg = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);

    uint32_t res = p6_op(idx1, arg);
    res ^= 0x5746;

    res = p6_op(idx2, res);
    values[0] = res + 0x9b24;

    count = 1;
    return write_answer(out, cap, values, count);
}
```

(4)自定义 seed 验证记录

先利用 ./defuser –seed 42 –phase 6 时得到 seed = 42 时的值，然后将该值与代入公式后计算得到的值比较，若相同则带入炸弹程序检测

(5)运行结果



7.secret

(1)关键汇编片段

检查输入字符串长度，拷贝字符串后校验前 4 字节触发标识，匹配成功则调用隐藏关主函数。

0x000000000040278c <+60>: call strlen

0x0000000000402791 <+65>: sub $0x24,%rax

0x0000000000402795 <+69>: cmp $0x7f,%rax

0x0000000000402799 <+73>: jbe 0x4027b0 <secret_gate+96>

0x00000000004027bb <+107>: cmpl $0x4e45504f,-0xb0(%rbp)

0x00000000004027c5 <+117>: jne 0x40279b <secret_gate+75>

0x00000000004027cd <+125>: call 0x402670 <secret_phase>

提取 context 最低 2 位，与输入的第一个数比对，不相等则校验失败。

```
0x00000000004026d0 <+96>:    mov    %r13d,%ecx
0x00000000004026d5 <+101>:   and    $0x3,%ecx
0x00000000004026d8 <+104>:   cmp    %ecx,-0x2c(%rbp)
0x00000000004026db <+107>:   jne    0x40269a <secret_phase+42>
```

计算两级异或公共参数与偏移常量，调用 secret_mix 完成核心运算，返回值与偏移量异或后得到第二个数的正确结果，与输入比对。

```
0x00000000004026e0 <+112>:   imul   $0xd515,%ecx,%r9d
0x00000000004026e7 <+119>:   shr    $0x20,%rsi
0x00000000004026ee <+126>:   shr    $0x20,%r8
0x00000000004026f2 <+130>:   xor    %r12d,%esi
0x00000000004026f5 <+133>:   xor    %r13d,%r8d
0x00000000004026f8 <+136>:   shr    $0x10,%r13
0x0000000000402701 <+145>:   xor    %r8d,%edi
0x0000000000402704 <+148>:   call   0x402650 <secret_mix>
0x0000000000402709 <+153>:   xor    %r9d,%eax
0x000000000040270c <+156>:   cmp    %eax,-0x28(%rbp)
0x000000000040270f <+159>:   jne    0x40269a <secret_phase+42>
```

组合位段常量计算中间值，与第二个数异或得到第三个数的正确结果，与输入第三个数比对，全部相等则通关。

```
0x0000000000402711 <+161>:   shr    $0x7,%esi
0x0000000000402714 <+164>:   lea    0xe16d(%r8,%rcx,1),%edx
0x000000000040271c <+172>:   add    %esi,%edx
0x000000000040271e <+174>:   xor    %edx,%eax
0x0000000000402720 <+176>:   cmp    -0x24(%rbp),%eax
0x0000000000402723 <+179>:   sete   %dl
```

secret_mix 核心运算：单输入基础位运算与算术运算组合：循环左移 7 位、加常量、异或常量后返回

```
0x0000000000402654 <+4>:     mov    %edi,%eax
0x0000000000402656 <+6>:     rol    $0x7,%eax
0x0000000000402659 <+9>:     add    $0xd515,%eax
0x000000000040265e <+14>:    xor    $0xe16d,%eax
0x0000000000402663 <+19>:    ret
```

(2)计算关系恢复与推理过程

首先通过分析 secret_gate 函数的汇编代码，程序会先检查输入字符串的长度，再拷贝字符串并校验首部的固定内容，结合小端字节序还原出触发标识，得出隐藏关需要输入带有指定前缀的字符串才能触发进入。进入 secret_phase 主函数后，程

序先调用 valid_u32_fields 校验输入格式，再通过 sscanf 解析整数，两处校验都要求输入包含 3 个整数，由此确定隐藏关的答案是 3 个 32 位无符号整数。第一个数的计算逻辑最简单，程序将 context 和 0x3 做按位与运算，只保留最低两位二进制，和第一个输入值比对，所以第一个数就是 context & 0x3，取值范围在 0 到 3 之间。接下来计算公共运算参数，先分别把 challenge 和 context 的高 32 位与低 32 位做异或，得到两个中间值，再将这两个中间值做一次异或，得到核心函数的输入参数。偏移常量的计算分为两步，先用第一个数乘以固定常量 0xd515，再和 context 右移 16 位的低 32 位值做异或，得到最终的偏移量。分析 secret_mix 核心函数的汇编可以还原出运算流程：输入值先循环左移 7 位，加上常量 0xd515，再和常量 0xe16d 异或后返回。核心函数的返回结果和偏移量异或之后，就是第二个数的正确答案。最后计算第三个数，将 challenge 高低异或的结果右移 7 位，加上 context 高低异或的结果、第一个数以及常量 0xe16d 得到中间值，再用第二个数和这个中间值异或，就得到了第三个数的正确答案。

(3)解法代码的实现

```c
static inline uint32_t rol7(uint32_t x) {
    return (x << 7) | (x >> 25);
}

static uint32_t secret_mix(uint32_t x) {
    return (rol7(x) + 0xd515) ^ 0xe16d;
}

int solve_secret(uint64_t challenge, uint64_t context, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;

    uint32_t v0 = (uint32_t)(context & 0x3);
    values[0] = v0;

    uint32_t chal_half = (uint32_t)(challenge >> 32) ^ (uint32_t)challenge;
    uint32_t ctx_half = (uint32_t)(context >> 32) ^ (uint32_t)context;
    uint32_t arg = chal_half ^ ctx_half;

    uint32_t offset = v0 * 0xd515;
    offset ^= (uint32_t)(context >> 16);
    uint32_t v1 = secret_mix(arg) ^ offset;
    values[1] = v1;

    uint32_t tmp = (chal_half >> 7) + ctx_half + v0 + 0xe16d;
    values[2] = v1 ^ tmp;

    count = 3;
    return write_answer(out, cap, values, count);
}
```

(4)自定义 seed 验证记录

用 42 作为 seed 进行验证，利用 defuser –secret 测试。按照逆向还原的计算逻辑分步计算，先提取 context 的最低两位得到第一个数，再依次算出公共参数、偏移常量，代入核心运算公式得到第二个数，最后组合位段运算得到第三个数。运行解题程序输出的三个结果分别为 0、419431831、1716941839，和计算的结果完全匹配。

(5)运行结果

```
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./defuser --s
eed 42 --secret
0 419431831 1716941839
hazel@LAPTOP-FBAT040H:/mnt/c/Users/lenovo/ics/ics-2026-bomblab$ ./bomb --seed
 42 --secret
--secret is a defuser-only mode.
```